Game runtime mechanisms

Read opening rules and only changed mechanism sections, reusing current sections in context; unchanged plumbing needs no new acceptance. Workflow owns scope and art/audio, delivery owns commands. Fast prototype does not add these mechanisms except requested multilingual support.

Engine mechanisms

Audio integration

Reuse the existing Music/SFX routing, gesture unlock and bounded voices, and keep empty routes safe across pause,
retry and scenes. New games that use audio keep the absolute base gains BGM 3× (+9.5424 dB) and SFX/UI 2×
(+6.0206 dB) over the inherited originals: place MusicBase/SfxBase before senders so each route crosses once,
replace old targets instead of stacking boosts, and keep the ⩽0 dB guards, saved settings and explicit mixes.
Silent games need no gain plumbing, and existing delivered mixes stay unchanged. Spatial audio keeps real source
positions; music and global HUD sounds are separate. When music is used, keep the shared browser BGM adapter and
its native fallback.

Browser-safe lifecycle

Bound and reuse voices, projectiles, particles, timers and subscriptions across pause, retry and scenes; change
pause or stream state on transitions, not every frame. Browser BGM uses the independent Web Audio renderer in
`scripts/manus/browser_bgm_player.gd`, never frame- or timer-fed PCM or finished-callback loops. It decodes once into
a bounded LRU AudioBuffer cache: call `prepare()` during loading or a transition before gameplay, then `play()`
reuses the buffer. Imported duplicates share an explicit original-path `buffer_key`, and changed PCM needs a new key.
Decode imported resources from the PCK, keep SFX in Godot and keep `thread_support=false`.
Route track, loop, position, pitch, mute, pause, duck, crossfade and Music/Master controls through the adapter.
External BGM does not inherit buses: the adapter mirrors player/bus volume,

AudioEffectAmplify gain,
AudioEffectLimiter makeup and AudioEffectHardLimiter pre-gain only. HardLimiter ceiling/release and nonlinear
limiting are not reproduced; the independent browser renderer does not share Godot's combined BGM/SFX peak
protection. Positive HardLimiter pre-gain can clip browser BGM without the limiter's peak protection.
Reserve mix headroom and never rely on a Godot Master limiter to protect browser BGM. Keep native
Godot and the guarded Web fallback (`PLAYBACK_TYPE_STREAM` for long BGM) without overlapping backends; pending
autoplay waits for valid input, and stale starts are guarded. Stop voices and release removed cues with
`release_cached_stream()` , evict only unreferenced tracks, keep OS/bfcache suspend and restore, and release the context
on real teardown. Native tests cannot prove browser continuity through
main-thread stalls, so report that as pending user acceptance.

Fast, recoverable startup

Keep the title and controls interactive with only the required fonts, UI, settings and small art. Defer scenes,
generation, media and decoding until needed, in bounded per-frame work; instantiate scenes on the main thread. Start
blocks duplicate transitions and reports recoverable content errors. Lazy PCK content reduces work, not transfer
bytes. Loading progress uses real bytes or phases and an indeterminate state for unknown totals; slow or stalled
notices are nonfatal, offer Retry, and clear on late success. When changing this mechanism, check cold load,
repeated Start, stall, unknown total, late success and Retry.

Animated backgrounds without startup blocking

Show a small static placeholder immediately and retain it as fallback. Load animations asynchronously after the
title is interactive, outside Start/preload dependencies; swap on a ready frame, guarding exited scenes and stale
completions.

Large explorable maps: presentation culling and live state

Offscreen simulation, pathfinding, orders, combat, economy, timers, replication, saves and minimap stay live. Cull
only cosmetics against current camera bounds, with sprite margins and separate fog; never remove gameplay actors,
projectiles or collision by distance. Reentry shows current state without replaying missed effects. Optimize measured
bottlenecks with existing spatial or dirty-region mechanisms; compare crowded, zoomed and revisited views to a
baseline rather than relying on empty-map FPS.

Destructible props

Destroyed rubble and fragments render below actors in effective world z while keeping footprint, collision release
and reward position; reset restores the intact state.

Requested multilingual support

Extend the selected template's localization system rather than adding a parallel one. Inspect its locale service,
catalogs and settings owner first: templates use different APIs, locale IDs and flat versus `{locale, entries}` JSON.
For example, Generic uses `autoload/i18n.gd` with `localization/en.json` and `zh-CN.json`; Tower Defense uses
`autoloads/i18n.gd`, `scripts/ui/components/ui_copy.gd` and `en-US.json` / `zh-CN.json`; Puzzle uses `scripts/core/localization.gd` and `en.json` / `zh_CN.json`. Follow the initialized project's actual paths and API.

Keep stable keys, complete translations and matching whole-message placeholders for every exposed language;
route title, tutorial, HUD, settings, results and dynamic copy through the existing lookup/formatting service.
Keep player-facing words live, not baked into art. Extend the existing title/settings selector and persistence owner;
restore explicit saved choices before supported browser/OS detection and a documented fallback. Normalize locale IDs
to the catalogs actually shipped. A single-language game does not need runtime language detection or a selector.
Refresh visible controls through the template's signal or rebuild path without resetting gameplay, progress or audio
settings. Keep the Web loader and saved game locale aligned where the template has that bridge.

Include every exposed catalog in the export. Add a fallback font per uncovered script: the matching Noto family (e.g. Noto Sans KR/JP/TC/Arabic) from
GitHub `google/fonts` (`ofl/<slug>/`), at the same weight in the template font resources, before Noto Sans SC for JP/TC so Han keeps regional forms.
Subset CJK to its script (all Hangul syllables; kana and JIS X 0208 kanji; Big5 level-1 hanzi). Put it in `assets/fonts/` (not `assets/template/`) with
that folder's `OFL.txt` unchanged as `<Family>-OFL.txt` , and add `assets/fonts/*-OFL.txt` to the Web preset's `include_filter` . Reuse the template's
localization checks for key/placeholder parity, switching before and during play, saved choice after reload, fallback, dynamic text and layout on
exported resources; font/boot checks alone do not prove translations work. Support the requested languages, not an automatic EN/CN-only pair.

Tweak presentation language

For new Godot games, generate Tweak parameter labels, descriptions, category names, option labels
and human-readable units in the session default working language supplied by Node through Blueprint planning.
Reuse `data.state.language` from the canonical `webdev.config` GET `game/blueprint` already required after setup
or recovery; read it once if absent from context. Do not request a separate language parameter or infer it from
the game's language, conversation text, or sandbox/browser/OS locale. This overrides genre guides' Tweak
localization rules. Preserve existing projects' Tweak localization, including newly added controls, unless the user
asks to change it.

If init explicitly reports structured `planning_unavailable` with no saved Blueprint, use the session default
working language already known to the agent; only when unknown, use English. Pin the new catalog's presentation
language independently of the game's locale. Do not retry planning just to obtain a language or block the authorized
development. An ordinary Blueprint read error is not this fallback signal; follow the existing-game recovery
instructions and preserve existing Tweak text.

Keep parameter IDs, option values, source defaults, bounds and apply timing unchanged. Prevent the game's locale
from overriding the catalog's presentation language, including transport-generated

description suffixes. This is
generation-time language selection; do not change the game's own language or add a player language selector or
live Manus-language switching. The Addon panel's fixed UI follows the current host UI locale; authored catalog
text retains its generated or existing language when the host locale changes, without automatic translation.

Development Tweak when needed

The Addon generates the floating frosted-glass Tweak popover from the game's descriptors; never create an in-game
panel, launcher or F10 shortcut. Adapt the existing catalog to the requested game, extend it for meaningful tuning
and remove controls with no real consumer (at most 128). Keep one parameter manager with stable IDs, typed defaults,
bounds/options, localized labels, honest apply timing and existing run eligibility. Edits/reset affect only the
Addon draft. Apply validates and commits the whole patch without disk writes or preview reload; unchanged values
emit nothing. Gameplay boundaries consume pending values but never send catalogs. Do not add numeric revisions.
Save with Manus sends selected values to the task for source edits and the normal build/checkpoint flow; it does
not publish. Keep the adapter in `scripts/manus/preview/`, which release excludes with its autoload entry. Normal
player Settings and gameplay consumers remain. Existing genre rules below identify the actual parameter owners.

Follow Tweak presentation language for the catalog and preview adapter.
For example, `tuning_transport.gd`'s `control_for` appends English timing text such as `Next Attack.` when mapping
apply modes. Localize that text in the project's adapter, preserving the actual apply boundary.

Preview tuning handoff

The Addon owns the generated Tweak panel. Apply only updates the resident preview. Save with Manus sends selected
values to the task: validate against the game catalog, edit the requested source defaults, then build and checkpoint.
Publication remains explicit. The installed genre Skill identifies the game's parameter

manager; no additional
transport document is required for ordinary game generation.

Three.js preview tuning

For a Three.js project with Tweak support, keep its existing game-owned parameter manager and
Vite preview plugin when adapting gameplay. The starter uses `src/game/tuning.ts` and `config.ts`;
register only controls with actual consumers and the appropriate action/run boundary. Save
snapshots update source defaults, then use the normal Web build and checkpoint workflow.
Do not add a second panel/store or expose client-only gameplay overrides in authoritative multiplayer. Preserve the per-run `unranked` latch and exclude tuned runs from local and online score
submission; online eligibility must also be checked by the authoritative server.
Older projects need explicit integration; do not scaffold over their existing code.

Release candidates and developer-UI exclusion

Release preparation freezes inputs and excludes `scripts/manus/preview/`, its autoloads and developer tests/tools/docs
from the candidate. Keep gameplay, defaults and player Settings independent of those modules. Keep provenance/licenses
in source; ignore scratch and hydrated media. Failed export preserves the last successful build.
Candidate commands and recovery follow delivery.